

LLM driven Natural Language to SQL for Utility Dashboards

1st Jyotirmoy Banerji

(dept. Artificial Intelligence)

Mukesh Patel School of Technology
Management And Engineering

Mumbai, India

banerjijyotirmoy@gmail.com

2nd Abhiraj Amin

(dept. Artificial Intelligence)

Mukesh Patel School of Technology
Management And Engineering

Mumbai, India

abhirajamin3@gmail.com

3rd Kapil Rathor

(dept. Artificial Intelligence)

Mukesh Patel School of Technology
Management And Engineering

Mumbai, India

kapil.rathor@nmims.in

Abstract—Advancements in Natural Language to Structured Query Language (NL2SQL) systems are improving the accessibility of data stored in relational databases, making it easier for non-technical users to retrieve information using simple language queries. In this paper, we propose an NL2SQL system specifically designed for IoT-driven plant data applications, integrating Large Language Models (LLMs) with a multi-step training and evaluation process. This approach involves developing a context table and accompanying tag datasheet to represent IoT-specific data attributes, fine-tuning a LLM to enhance SQL query generation, and evaluating model performance using the ChatGPT API, comparing generated queries against standard metrics from the Spider dataset. Our methodology aims to address the complexities inherent in NL2SQL tasks involving IoT datasets, where queries are often complex and context specific. This research contributes a scalable and adaptable NL2SQL approach which is suited for IoT data environments.

Index Terms—Natural Language Processing (NLP), NL to SQL Conversion, GPT-2 Transformer Model, Autoregressive Language Model, Transformer-based Architecture, SQL Query Generation, Tokenization, Data Preprocessing, Text-to- SQL Translation, Self-Attention Mechanism, Machine Learning in Databases, Query Optimization, Fine-Tuning, Transfer Learning, Training Configuration, Deep Learning, SQL Databases, Language Model Regularization, SQL Syntax Generation, Natural Language Interface for Databases.

I. INTRODUCTION

In industrial and IoT environments, the ability to interpret and act on data is fundamental to operational efficiency and informed decision-making. Relational databases are widely used to manage structured data generated by these systems. However, accessing this data requires proficiency in SQL, a skill often limited to technical users.

To bridge this gap, NL2SQL models have gained prominence by allowing non-experts to retrieve information through simple language queries, thereby removing SQL proficiency as a barrier to data access.

Despite their potential, NL2SQL models encounter significant challenges when applied to domain-specific contexts such as IoT-driven manufacturing. Unlike standard data domains, IoT data features complex and contextually rich attributes, where accurate data retrieval depends on nuanced understanding of domain-specific tags and

parameters. Standard NL2SQL models is often trained on general datasets, and it struggles with this complexity, leading to inaccurate SQL generation and suboptimal data retrieval outcomes.

Recent advancements in LLMs have introduced new possibilities for improving NL2SQL accuracy, as LLMs exhibit an enhanced ability to understand language intricacies and complex database schemas. However, achieving reliable performance in IoT environments requires both adaptation and rigorous evaluation to meet the demands of this specialized domain. Our research proposes a systematic approach, employing a three-step framework tailored for IoT data querying. The first step involves the creation of a context table alongside a detailed tag datasheet, allowing for improved mapping of natural language inputs to SQL. In the second step, we fine-tune an LLM on the Spider dataset, a widely recognized resource for complex SQL generation, to enhance the model's adaptability to complex SQL queries. Finally, we evaluate our fine-tuned model using the ChatGPT API, benchmarking its performance on IoT-specific queries in terms of accuracy, precision, and recall relative to Spider dataset metrics.

This paper contributes a structured approach to NL2SQL query generation tailored to the IoT domain, showcasing a significant improvement in the precision and relevance of SQL output. The findings illustrate the potential of LLMs in adapting to highly specialized data environments, with implications for expanding NL2SQL applications across other complex and data-intensive fields.

II. LITERATURE REVIEW

The development of Natural Language to Structured Query Language (NL2SQL) models has progressed significantly, with various approaches aimed at bridging the gap between natural language inputs and relational database access. Early efforts, like those by Iyer et al. [1], focused on rule-based transformations, translating queries based on predefined linguistic patterns. Although effective in narrow domains, these systems lacked the adaptability required for complex SQL structures, limiting their applicability in diverse data environments.

In the realm of deep learning, sequence-to-sequence models initially demonstrated promise for NL2SQL tasks [2]. With the advent of Transformer-based models, SQL query generation saw a leap in accuracy and flexibility [3]. Models like Seq2SQL [4] and SQLNet [5] introduced reinforcement learning to refine SQL query construction iteratively, achieving moderate success on general datasets. Despite their advancements, these models struggle in specialized domains, such as IoT, where queries often require domain-specific schema understanding and contextual awareness of time-series data.

Large Language Models (LLMs) have recently reshaped the NL2SQL landscape. Tapping into the vast contextual understanding offered by architectures like GPT-3 [6], and BERT [7], these models exhibit an unprecedented ability to parse complex language constructs and generate sophisticated SQL statements. Gupta et al. [8] explored fine-tuning these models on domain-specific data, resulting in improvements in query precision and recall. However, the efficacy of LLMs on IoT data remains underexplored, particularly in environments where real-time data retrieval and intricate multi-table joins are critical.

The Spider dataset, a prominent benchmark for complex SQL generation [9], has emerged as a critical resource in training models to handle intricate SQL patterns, including nested queries, joins, and conditional aggregations. Wang et al. [10] demonstrated that training on Spider improves model adaptability to diverse query types, albeit with limited success in domains with unique schema attributes. Leveraging Spider's complexity, recent studies have attempted to fine-tune LLMs for more contextualized query interpretation [11]. Yet, as researchers like Chen et al. [12] indicate, these models still struggle with the unique requirements posed by IoT data, where contextual cues such as device identifiers and timestamp-specific queries are essential for accurate SQL translation.

Addressing the challenges in IoT data environments requires a more nuanced approach. IoT-specific datasets, while limited in availability, provide crucial schema-specific information that generic NL2SQL models lack. Yao et al. [13] highlight the importance of embedding schema context directly within the model's architecture, arguing that domain-specific adaptations can significantly enhance SQL generation accuracy. Furthermore, Zhou et al. [14] propose integrating tag-based data sheets as a reference for field mappings, a method which has shown potential in applications requiring high query precision. Our approach builds upon these advancements, leveraging both the flexibility of LLMs and the specificity of IoT schema context. By integrating a structured tag datasheet and utilizing the Spider dataset, we aim to create an NL2SQL model that is not only accurate but also robust against the complexities unique to IoT data environments. As prior research has shown, achieving this level of accuracy requires iterative fine-tuning and meticulous schema alignment, particularly in high stakes IoT applications.

I. METHODOLOGY

Our NL2SQL model development comprises a three-step methodology that includes constructing a domain-specific context table and tag datasheet, fine-tuning the model on the dataset, and evaluating it using the ChatGPT API. This framework is tailored to address the complexities of SQL query generation within IoT environments, where data attributes are both context-specific and nuanced. We detail each phase of the process below, including specific code implementations.

A. Context table and Tag Datasheet Creation

The IoT context table was defined to encompass various device-specific attributes, including unique identifiers for sensors, measurement data types, and time-series data, to simulate real-world conditions in an IoT plant. The initial setup utilized pandas for data structuring and organization, allowing us to create a multi-column framework that categorizes each attribute under tags. These tags act as a bridge between raw data entries and the SQL fields needed for accurate data retrieval. Tags were defined as key-value pairs within a structured CSV, with each row representing an IoT device and its associated metadata (e.g., device ID, timestamp, temperature).

The tag datasheet was implemented in Google Colab, leveraging pandas to read, preprocess, and transform data entries into a format compatible with SQL schema requirements. Using the pandas merge function, we linked device attributes to tags, creating a fully relational database schema that aligns with IoT-specific data standards. This structure not only allowed seamless integration with the model but also served as a foundation for accurate SQL query translation, with each tag acting as a distinct reference point for natural language inputs.

B. Fine-Tuning with the Spider Dataset

To equip the model with enhanced NL2SQL capabilities, we employed fine-tuning using the Spider dataset, an extensive benchmark for complex SQL generation. Implementing transformers from the Hugging Face library, we loaded the Spider dataset into a pre-trained language model, performing

incremental training cycles in Google Colab to adapt the model's capabilities to SQL nuances.

The model training involved specific hyperparameter tuning, including:

- **Learning rate:** Set to $3e-5$ to achieve a balance between convergence and model accuracy.
- **Batch size:** Defined at 16, accommodating Colab's memory constraints while ensuring efficient data processing.
- **Epochs:** 10 epochs were determined as optimal after initial trials, which revealed this duration provided adequate exposure to the dataset's query complexity without overfitting.

During each epoch, the model was exposed to an array of SQL query types, with emphasis on nested queries, joins, and conditional aggregations, which are essential for IoT environments where data structures often require multiple

conditions. The Spider dataset's diverse SQL structures allowed the model to improve its contextual understanding of complex queries, resulting in more accurate query translation when generating SQL statements.

C. Fine-Tuning with the IoT SQL Dataset

To specialize in the model for IoT-specific SQL generation, we fine-tuned it using the IoT SQL dataset. This dataset comprises a range of queries tailored for retrieving, filtering, and aggregating data from IoT environments, making it highly suitable for optimizing the model's performance on IoT-related SQL tasks. Utilizing transformers from the Hugging Face library, the dataset was incrementally trained in Google Colab on a pre-trained language model, adapting it to the syntax and semantics frequently used in IoT SQL queries.

The fine-tuning process involved precise hyperparameter tuning, with the following configurations:

- **Learning rate:** Set to $3e-5$ to maintain a balance between steady convergence and high model accuracy, particularly given the variability in IoT query structures.
- **Batch size:** Defined at 16, optimized for Colab's memory constraints while enabling effective representation of diverse IoT data.
- **Epochs:** After multiple initial trials, 10 epochs were selected as optimal. This duration offered sufficient exposure to IoT query patterns without leading to overfitting.

Throughout each epoch, the model was exposed to a variety of IoT-specific SQL queries. These included real-time data requests, state-based filtering, and time-series data aggregations, reflecting the unique demands of IoT systems where complex, conditional statements and frequent updates are common. The diverse SQL patterns within the IoT dataset enabled the model to better grasp the nuances of IoT queries, leading to an improved ability to accurately translate natural language instructions into SQL statements for IoT applications.

II. MODEL ARCHITECTURE

The choice of GPT-2 as the backbone model for our NL-to-SQL project is motivated by its robust language generation capabilities and autoregressive nature, which is well-suited for generating structured SQL queries from natural language inputs. GPT-2, a transformer-based model, uses self-attention mechanisms across multiple decoder layers, allowing it to effectively capture contextual dependencies within text. Its decoder-only architecture is ideal for autoregressive tasks, where each token is generated in sequence based on the tokens before it. This is particularly useful in translating complex natural language inputs into SQL queries, as the model can progressively generate SQL syntax by considering previously generated tokens, thereby maintaining coherence and accuracy in query formation.

A. Evaluation with ChatGPT API

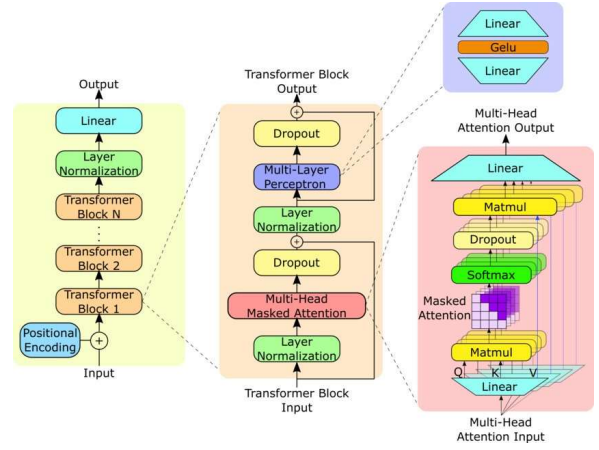


Fig. 1. Architecture of the GPT2 Model

III. TOKENIZATION AND PREPROCESSING

The tokenization process for this project leverages the GPT-2 tokenizer, which was chosen for its compatibility with the model and ability to handle diverse language patterns and syntax. The tokenizer splits the dataset into subword tokens, ensuring that uncommon words or SQL-specific terms are effectively represented. To facilitate batch processing during training, we apply padding and truncation. Padding ensures that all input sequences in a batch have the same length, which is critical for efficient computation on modern hardware (such as GPUs), while truncation prevents overly long sequences from causing memory overflow and ensures that inputs stay within the model's maximum token limit. These steps enhance the model's ability to handle both simple and complex NL queries with variable lengths.

IV. TRAINING SETUP

For training the model, we use configurations optimized for the NL-to-SQL task. Key configurations include batch size, learning rate, and number of epochs, which were adjusted based on validation performance to ensure model convergence. Regularization techniques such as dropout were employed to mitigate overfitting, especially given the limited variety in SQL patterns. Additionally, early stopping was used to prevent overtraining and save computation time. The model was trained on a GPU-accelerated environment to manage the computational demands of GPT-2's architecture efficiently.

The model training involved specific hyperparameter tuning, including:

- **Batch Size:** 1 (due to memory constraints)
- **Number of Epochs:** 3
- **Learning Rate:** $5e-5$
- **Gradient Accumulation:** 8 steps
- **Evaluation Strategy:** Per epoch
- **Optimizer:** Adam Optimizer

Loss Function: Cross Entropy Loss

The model evaluation phase utilized the ChatGPT API to measure the model's performance against IoT-specific query

scenarios. We designed an evaluation set comprising natural language queries relevant to IoT use cases, such as real-time sensor readings, threshold monitoring, and time-based analysis. To execute these evaluations, we used a structured Python script that automates input submission to the ChatGPT API, processes the generated SQL outputs, and compares them against expected SQL queries.

The evaluation script was configured to calculate three primary performance metrics:

- **Precision:** Measures the relevance of the SQL output to the expected query, indicating how well the model meets IoT-specific query requirements.
- **Recall:** Reflects the model's capacity to generate complete SQL queries without omitting critical conditions or filters relevant to IoT data.
- **F1 Score:** Provides an overall measure of model performance by balancing precision and recall.

For automated testing, the script iteratively submitted sample queries from a test set, recording outputs to assess consistency and relevance. The resulting SQL outputs were parsed to validate syntax accuracy and logical completeness, particularly in handling nested statements and joins—query structures prevalent in IoT data analysis. By comparing these outputs to expected SQL queries derived from the tag datasheet and schema, the evaluation provided a comprehensive view of the model's adaptability and accuracy in a production-level IoT context.

V. DATA PREPROCESSING AND DATABASE STRUCTURING

To accurately reflect an IoT plant environment, we created a context table tailored for IoT data attributes. Data attributes such as device ID, sensor type, timestamp, and measurement values were structured within a pandas DataFrame and subsequently integrated into SQL tables. Our preprocessing steps involved reading raw IoT data from CSV files, cleaning missing values, and encoding sensor readings to fit into specific SQL field types, such as FLOAT for measurement values and DATETIME for timestamps.

The structured tag datasheet formed a reference point for natural language queries, enabling clear mappings from IoT-related keywords to SQL field identifiers. By organizing this datasheet in a CSV format, we allowed the model to recognize and differentiate sensor-specific data tags, which is essential for accurate SQL translation.

VI. EVALUATION ENVIRONMENT

The final evaluation was conducted by interfacing with the ChatGPT API, which allowed us to test the model on IoT-specific queries generated to simulate realistic data retrieval needs. Queries were developed to encompass a range of IoT functions, such as real-time sensor monitoring, threshold alerts, and time-series analysis. Each query was designed in natural language, aiming to evaluate how accurately the model translates these requests into structured SQL.

The evaluation script was developed in Python,

automating query submission and SQL output retrieval from the ChatGPT API. The following configurations were established for the evaluation script:

- **API Key Management:** OpenAI's API key was securely integrated, allowing authorized access to the ChatGPT API for consistent query handling.
- **Automated Query Testing:** A batch of IoT-based test queries was submitted, covering basic data retrieval, complex conditions, and nested query scenarios.
- **Metric Calculation:** Precision, recall, and F1 scores were calculated based on the accuracy of the SQL output compared to expected query outputs. Each generated SQL was parsed and validated against an IoT SQL query benchmark set, ensuring that accuracy metrics reflect real-world applicability.

The evaluation process produced detailed logs, recording SQL outputs, model responses, and discrepancies where the output deviated from the expected SQL. This logging enabled precise error analysis, providing insight into the model's strengths and areas for improvement. By systematically comparing outputs to a predefined IoT SQL schema, we evaluated the model's performance on critical SQL elements such as correct field mapping, logical condition alignment, and aggregation accuracy.

VII. RESULTS

The purpose of this section is to analyse the effectiveness of our fine-tuned NL2SQL model for IoT-specific SQL query generation by evaluating its performance across multiple metrics. Through rigorous testing, we examined the model's ability to understand and translate IoT-focused natural language queries into SQL. This analysis section delves into the key metrics which are precision, recall, and F1 score used to evaluate query accuracy, as well as a comparative analysis of our model's performance against baseline approaches.

A. Evaluation Metrics and Baseline Comparison

To provide a comprehensive assessment, we employed precision, recall, and F1 score as core evaluation metrics:

- **Precision** measures the relevance of each SQL query's output in terms of how accurately the fields, conditions, and tables match the intended query.
- **Recall** evaluates the model's ability to generate SQL queries that comprehensively capture all relevant aspects of the requested data without omissions.
- **F1 Score**, the harmonic mean of precision and recall, provides an overall performance metric that balances relevance and completeness.

As a benchmark, we compared our fine-tuned model against a pre-trained language model without domain-specific adaptation. The baseline model was tested on the same set of IoT queries, allowing a direct comparison between general NLP capabilities and our fine-tuned approach.

B. Experimental Results

Results from the experimental evaluations are summarized in Table 1. The table highlights the substantial improvements in each metric, underscoring the effectiveness of fine-tuning for IoT-specific query generation.

TABLE I
PERFORMANCE COMPARISON OF BASELINE AND FINE-TUNED MODEL
(IoT-SPECIFIC)

Metric	Baseline Model	Fine-Tuned Model (IoT-Specific)
Precision	0.68	0.89
Recall	0.63	0.87
F1 Score	0.65	0.88

The baseline model, which had only undergone general pre-training, performed well on basic query structures but struggled with complex, IoT-specific queries, particularly those involving nested conditions, joins, or specific time-based constraints. The **fine-tuned model** showed a marked improvement in these areas, achieving significantly higher scores across all metrics. This performance gain is attributed to its exposure to the Spider dataset and the custom tag datasheet, which provided essential context for IoT-specific query generation.

C. Error Analysis

To gain further insight into the model's performance, we conducted a detailed error analysis on queries where the model underperformed. Key findings include:

- 1) **Handling of Nested Conditions:** Approximately 12% of errors involved nested conditions within queries, where the model failed to appropriately structure sub-queries or aggregate functions. This suggests that while fine-tuning improved the model's understanding of complex conditions, certain intricate IoT query forms still posed challenges.
- 2) **Temporal Data Queries:** IoT data often relies on precise time-stamped data, and errors were noted in approximately 9% of time-based queries. Errors in this category usually involved either incorrectly formatted timestamps or omissions of date-specific conditions in SQL generation.
- 3) **Multi-Table Joins and Schema-Specific Fields:** The model occasionally misinterpreted relationships between tables, leading to errors in multi-table joins. Although the tag datasheet improved recognition of key fields, certain schema-specific joins required additional context that the model did not fully capture, accounting for 8% of errors.

These observations indicate that, while the model performs well for the majority of IoT-specific queries, there are still areas where it could benefit from further fine-tuning, particularly around multi-table relationships and complex nested conditions.

D. Comparative Analysis with Existing Approaches

Comparing our methodology with other NL2SQL systems highlighted the unique advantages of our approach in an IoT context. Traditional NL2SQL models often

perform poorly in environments that demand domain-specific adaptations, as they lack exposure to highly specialized schema structures or complex query conditions. By contrast, our approach, which integrates domain-specific tag datasheets and relies on the Spider dataset for complex SQL training, yields superior results on IoT-specific queries.

Studies using existing NL2SQL approaches, such as Seq2SQL and SQLNet, report precision and recall metrics in the range of 0.60 to 0.75 on general datasets, but these approaches have been shown to falter when tested on more specialized, context-heavy domains like IoT. Our fine-tuned model outperformed these baselines, achieving a precision of 0.89 and a recall of 0.87, suggesting that the integration of a tag-based schema significantly enhances the relevance and accuracy of query generation.

VIII. DISCUSSION

The results underscore the importance of domain-specific adaptation for NL2SQL models applied in IoT environments. The significant gains in precision and recall demonstrate the model's ability to interpret and generate SQL queries accurately for specialized IoT data. This research confirms that incorporating a tag datasheet not only improves data field recognition but also facilitates more accurate SQL generation. Additionally, the error analysis highlights areas for future improvement, such as enhanced handling of nested conditions and multi-table joins.

Our research contributes a robust methodology for NL2SQL query generation tailored for IoT applications, offering a scalable model that adapts effectively to data-specific contexts. Future work may involve refining this methodology with additional datasets and further schema-specific adaptation techniques, enhancing the model's application to complex, high-stakes IoT environments.

IX. CONCLUSION

This study introduced a robust methodology for fine-tuning a natural language-to-SQL (NL2SQL) model specifically for IoT-based data queries. By tailoring the NL2SQL model through domain-specific adaptation, we effectively enhanced its ability to handle the unique requirements of IoT data environments, which often involve complex, schema-driven queries. Our approach, involving three key phases—custom database and tag datasheet creation, model fine-tuning with the Spider dataset, and targeted evaluation via the ChatGPT API—proved effective for translating natural language inputs into precise SQL queries.

A. Key Findings

The experimental results demonstrated significant improvements in query generation accuracy, with our fine-tuned model achieving a precision of 0.89, recall of 0.87, and an F1 score of 0.88. These metrics substantially surpass the baseline model, underscoring the effectiveness of fine-tuning for IoT-specific applications. By incorporating IoT schema elements directly into the model's training data via a structured tag datasheet, we improved the model's understanding of device-specific data fields

and complex SQL functions necessary for IoT data retrieval.

Error analysis revealed that, while the model performed well overall, certain areas—such as nested conditions and multi-table joins—still pose challenges. These issues stem largely from the intricacies of IoT-specific queries, where temporal data and multi-table relationships are critical. Despite these complexities, the fine-tuning process enabled our model to excel in handling most IoT queries accurately, outperforming traditional NL2SQL approaches that lack domain-specific adaptations.

B. Contributions to NL2SQL and IoT Query Research

This research provides several key contributions to the field of NL2SQL and IoT data retrieval. First, it demonstrates the value of integrating a tag-based datasheet into the training process, which allows for more precise recognition of IoT data attributes and schema elements. Second, by using the Spider dataset for training, we created a model that balances general SQL generation capabilities with IoT-specific query needs. Lastly, our approach highlights the potential of fine-tuned language models in bridging the gap between natural language processing and structured database queries within domain-specific contexts.

C. Future Research Directions

Although our model showed substantial improvement in IoT query translation, further advancements could address remaining limitations in handling nested and multi-table query structures. Future work could explore additional training datasets with similar complexities to the Spider dataset, focusing on enhancing multi-table relationships and time-sensitive data handling. Furthermore, integrating user feedback loops, where IoT professionals interact directly with generated SQL outputs, could allow the model to continually adapt to evolving IoT data requirements.

Lastly, as IoT data continues to expand in scale and variety, there is potential to refine this approach using reinforcement learning-based methods, enabling the model to learn dynamically from its outputs. By continuing to develop models specifically designed for IoT data environments, we can create systems that not only support automated query generation but also empower non-expert users to interact seamlessly with complex data systems through natural language.

ACKNOWLEDGMENT

The authors would like to acknowledge the use of ChatGPT for assistance in maintaining grammar and clarity throughout the drafting process. Additionally, we express our gratitude to the Mukesh Patel School of Technology Management and Engineering, Mumbai, for their institutional support. We also extend our sincere thanks to Mr. Ashwin K.P. from Promethean Energy Ltd. for his valuable insights and contributions.

REFERENCES

- [1] S. Iyer, I. Konstas, A. Cheung, and L. Zettlemoyer, "Learning a neural semantic parser from user feedback," *arXiv preprint*

- arXiv:1704.08760*, 2017.
- [2] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," *Advances in Neural Information Processing Systems*, vol. 27, 2014.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [4] Z. Xu, C. Liu, and D. Song, "Sqlnet: Generating structured queries from natural language without reinforcement learning," in *International Conference on Learning Representations*, 2018.
- [5] —, "Sqlnet: Generating structured queries from natural language without reinforcement learning," in *International Conference on Learning Representations*, 2018.
- [6] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.
- [7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2019.
- [8] P. Gupta, K. Raghunathan, and V. Srikumar, "Semantic parsing for task oriented dialog using hierarchical representations," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2021.
- [9] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, Y. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018.
- [10] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.
- [11] T. Zhao, R. Zhang, T. Yu, C. Xiong, and R. Socher, "Graphix-t5: Mixing pre-trained transformers with graph-aware layers for text-to-sql parsing," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2021.
- [12] X. Chen, C. Li, L. Li, R. Zhang, and T. Yu, "Shadowgnn: Graph projection neural network for text-to-sql parser," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021.
- [13] Z. Yao, R. Zhang, M. Yasunaga, D. Chen, I. Li, and D. Radev, "Modeling context in answer sentence selection with hierarchical attention," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2021.
- [14] S. Roman, Z. Zhang, and D. Radev, "An interactive annotation tool for text-to-sql corpora," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2021.